

Livrable Développement d'applications distribuées

CESI 
ÉCOLE D'INGÉNIEURS



Développement d'applications distribuées

Réalisé par

Lisa ACHOUR

Allan BOUHAMED

Thais VIANES

Projet encadré par

Djamel AOUAM, Ali SKAF & Mathieu DESHORS

Livrable - Projet DAD - 25/06/2026

Synthèse Managériale

Ce document présente le bilan technique et fonctionnel de Breezy, réseau social de microblogging développé en projet collaboratif d'ingénierie informatique (A3) et mené jusqu'à une mise en production réelle en HTTPS (breezy.allanbouhamed.fr).

Double objectif pour ce livrable : fournir une application sociale complète et sécurisée tout en bâtissant une architecture microservices évolutive, capable d'accueillir de nouvelles fonctionnalités sans refonte du code métier.

Breezy respecte l'intégralité du cahier des charges (comptes vérifiés, posts de 280 caractères, fil d'actualité, likes, réponses imbriquées, abonnements, notifications, recherche, thème clair/sombre) et va nettement au-delà : messagerie privée, sondages, reposts, bookmarks, profils privés, OAuth Google/GitHub, modération à sanctions réversibles et internationalisation en 6 langues.

Le découpage en microservices (1 passerelle + 8 services) sur une base PostgreSQL à 6 schémas garantit une base évolutive et maintenable. L'authentification (JWT HS256, jeton de rafraîchissement rotatif et haché, bcrypt), le contrôle d'accès par rôles (RBAC) et la validation systématique des entrées assurent un bon niveau de sécurité.

Le projet a été livré dans les délais (≈ 3 semaines, ≈ 416 commits coordonnés par Gitflow), avec une chaîne CI/CD GitHub Actions et une conteneurisation Docker garantissant des déploiements fiables et reproductibles.

En conclusion, Breezy constitue un socle stable, sécurisé et déjà déployé en production, prêt pour les prochains jalons (temps réel, scalabilité, tests de bout en bout). Les principaux axes de consolidation identifiés sont le renforcement de la couverture de tests et le durcissement de la sécurité.

Liste de diffusion

Ce tableau répertorie tous les destinataires ayant accès au document :

Entreprise / Organisme	Destinataire(s)	Contacts	Rôle
Breezy	Equipe Breezy	support.breezy@googlegroups.com	Dev Breezy
CESI	DESHORS Mathieu	mat@kodz.fr	Responsable Projet
CESI	AOUAM Djamel	daouam@cesi.fr	Directeur informatique
CESI	SKAF Ali	askaf@cesi.fr	Responsable informatique

Sommaire

Développement d'applications distribuées.....	2
Synthèse Managériale.....	3
Liste de diffusion.....	3
Sommaire.....	4
1. Introduction.....	6
1.1 Contexte, problématique et enjeux.....	6
1.2 Objectifs et périmètre.....	7
1.3 Plan du document.....	7
2. Cadrage produit et gestion de projet.....	8
2.1 Présentation de Breezy.....	8
2.2 Organisation de l'équipe et méthode.....	8
2.3 Gestion des risques.....	9
3. Analyse des besoins.....	10
3.1 Besoins fonctionnels.....	10
3.2 Matrice des rôles et permissions.....	10
3.3 Besoins non fonctionnels et priorisation MoSCoW.....	11
4. Architecture et conception (partie technique cœur).....	12
4.1 Architecture microservices : monorepo, 9 binaires, 1 module Go / 1 image Docker..	13
4.1.1 Rôle de l'API Gateway et routage conditionnel.....	13
4.1.2 Patron interne d'un service.....	15
4.2 Modèle de données : PostgreSQL à 6 schémas.....	17
4.3 Conception de l'API REST et niveaux d'accès.....	18
4.4 Conception UX/UI : responsive mobile-first, thème clair/sombre, i18n 6 langues.....	20
5. Choix technologiques et écarts au sujet.....	21
5.1 Choix techniques retenus.....	21
5.1.1 Le choix back-end.....	21
5.1.2 Le choix front-end.....	21
5.1.3 Le choix du déploiement.....	22
5.2 Justification des écarts avec le sujet initial.....	22
6. Réalisation / Développement.....	23
6.1 Vue D'ensemble.....	23
6.2 Fonctionnalités Obligatoires.....	24
6.3 Fonctionnalités Au-delà Du Sujet.....	25
6.4 Implémentations Phares.....	25
6.4.1 Réponses Imbriquées Et Sondages.....	25
6.4.2 Sanctions Réversibles.....	26
6.4.3 Chiffrement de bout en bout des messages privés.....	26
Génération et publication des clés.....	26
Chiffrement côté client.....	27
Stockage côté serveur.....	27

Compatibilité rétroactive.....	27
7. Sécurité.....	28
7.1 Authentification.....	28
7.2 Autorisation RBAC, bcrypt et politique de mot de passe forte.....	29
7.3 Validation, CORS, injections — et limites assumées (pas de rate limiting ni CSP)...	30
7.4 Tests unitaires et couverture de code.....	32
8. Qualité, DevOps et déploiement.....	34
8.1 Qualité et tests : stratégie, lint, revue de code (CODEOWNERS) et dette technique assumée.....	34
8.2 CI/CD GitHub Actions et conteneurisation Docker multi-stage.....	36
8.3 Déploiement en production : VPS, nginx, HTTPS Let's Encrypt, seed de démo.....	37
8.3.1 Hébergement sur serveur privé virtuel (VPS).....	37
8.3.2 Serveur web et reverse proxy (Nginx).....	38
8.3.3 Chiffrement et certificats (HTTPS / Let's Encrypt).....	38
8.3.4 Initialisation des données de démonstration.....	38
9. Bilan et perspectives.....	39
9.1 Résultats au regard des objectifs et compétences mobilisées.....	39
9.2 Difficultés rencontrées et compromis assumés.....	39
9.3 Axes d'amélioration (temps réel WebSocket, scalabilité, tests e2e, durcissement sécurité, licence).....	39
10. Conclusion.....	40
11. Bibliographie et webographie.....	42
1. Langages et Frameworks.....	42
2. Architecture et Infrastructure.....	42
3. Sécurité et Protocoles.....	42
12. Annexes.....	43

1. Introduction

1.1 Contexte, problématique et enjeux

Le projet Breezy a été réalisé dans le cadre du module développement d'applications distribuées. L'objectif était de concevoir une application web de type réseau social permettant aux utilisateurs de publier de courts messages, d'interagir avec d'autres membres et de gérer leur profil.

Le projet devait être réalisé sur une période d'environ trois semaines par groupe de quatre étudiants. Cette contrainte de temps a nécessité la mise en place d'une organisation rigoureuse afin de développer un produit fonctionnel tout en respectant les exigences du sujet.

L'un des principaux enjeux du projet était la conception d'une architecture distribuée capable de séparer les différentes responsabilités de l'application. Pour répondre à cette problématique, une architecture orientée microservices a été mise en œuvre. Cette approche permet d'isoler les fonctionnalités principales telles que l'authentification, la gestion des utilisateurs, les profils ou encore les publications.

Le projet devait également répondre à plusieurs contraintes techniques, notamment la sécurisation des accès grâce aux jetons JWT, la persistance des données dans une base PostgreSQL, la communication entre services ainsi que la conteneurisation de l'ensemble de l'application avec Docker afin de faciliter son déploiement et son exécution.

1.2 Objectifs et périmètre

L'objectif principal du projet est de développer une plateforme de microblogging permettant aux utilisateurs de créer un compte, s'authentifier, publier des messages et interagir avec les autres membres de la plateforme.

Les fonctionnalités principales réalisées comprennent la création et l'authentification des comptes utilisateurs à l'aide de JWT, la gestion des profils, la publication et la consultation de messages, ainsi qu'un système d'interactions reposant sur les likes et les abonnements entre utilisateurs. Le projet intègre également des fonctionnalités de modération des contenus et s'appuie sur une architecture en microservices centralisée par une API Gateway.

Au-delà des fonctionnalités obligatoires, le projet vise également à mettre en œuvre une architecture moderne reposant sur plusieurs services indépendants développés en Go, communiquant à travers une passerelle d'API. Cette organisation facilite l'évolution future de l'application et améliore sa maintenabilité.

Le périmètre du projet comprend le développement du back-end en Go avec Gin et GORM, la base de données PostgreSQL, le front-end en Angular ainsi que la conteneurisation de l'ensemble des composants avec Docker.

1.3 Plan du document

Ce rapport présente le contexte et les objectifs du projet Breezy, puis l'analyse des besoins et les choix de conception réalisés. Il détaille ensuite l'architecture microservices, les technologies utilisées, les fonctionnalités développées ainsi que les mécanismes de sécurité mis en œuvre. Enfin, il aborde les aspects liés à la qualité, au déploiement et se conclut par un bilan du projet et ses perspectives d'évolution.

2. Cadrage produit et gestion de projet

2.1 Présentation de Breezy

Breezy est une application de microblogging inspirée des plateformes de type Twitter/X, permettant aux utilisateurs de publier et d'interagir via des messages courts. Le projet vise à proposer une alternative légère, performante et modulaire, adaptée à un contexte de développement distribué.

La cible principale correspond aux utilisateurs souhaitant partager rapidement de l'information et interagir au sein d'une communauté en ligne. Le positionnement de Breezy repose sur une architecture technique moderne et évolutive, orientée microservices, afin de garantir la scalabilité et la maintenabilité de la solution.

2.2 Organisation de l'équipe et méthode

Le projet a été réalisé en équipe avec une organisation structurée afin de répartir efficacement les responsabilités et d'assurer un développement parallèle des différentes briques applicatives. Chaque membre s'est vu attribuer des périmètres fonctionnels distincts, principalement entre le backend et le frontend, afin de limiter les dépendances et faciliter l'intégration globale du système.

La gestion du code a été assurée via Git en s'appuyant sur une approche inspirée de Gitflow. Cette méthode permet de distinguer les différentes étapes de développement à travers plusieurs branches (développement, fonctionnalités et intégration), garantissant ainsi un meilleur contrôle des versions et une stabilité du code principal.

Un fichier CODEOWNERS a également été mis en place afin d'automatiser l'attribution des responsabilités sur les différentes parties du dépôt. Ce dispositif facilite les revues de code et renforce la qualité des contributions en imposant une validation des modifications par les responsables concernés.

Le projet repose sur un total de 416 commits, reflétant une démarche de développement incrémentale et continue tout au long de la réalisation. Un planning de travail a été défini en amont afin de structurer les différentes phases du projet et d'assurer une progression cohérente des fonctionnalités jusqu'à l'intégration finale.

2.3 Gestion des risques

Plusieurs risques ont été identifiés dès le début du projet. Le premier concerne la contrainte de temps, relativement courte au regard de la complexité d'une architecture distribuée. Ce risque a été limité grâce à une priorisation des fonctionnalités essentielles et une organisation itérative du développement.

Un second risque important concerne l'intégration entre le front-end et le back-end, notamment dans un contexte de microservices. Pour y répondre, une API Gateway a été mise en place afin de centraliser les échanges et simplifier les appels côté client.

Enfin, la sécurité constitue un enjeu majeur du projet, en particulier pour la gestion des authentifications et des accès utilisateurs. L'utilisation de JWT ainsi que la structuration des services ont permis de réduire les risques liés aux accès non autorisés et à la manipulation des données.

3. Analyse des besoins

3.1 Besoins fonctionnels

Le projet Breezy répond à un ensemble de besoins fonctionnels structurés autour d'un système de microblogging. L'objectif principal est de permettre à des utilisateurs de créer un compte, de se connecter de manière sécurisée et d'interagir au sein d'une plateforme sociale.

Les fonctionnalités mises en œuvre couvrent l'ensemble du périmètre attendu, incluant la création et l'authentification des comptes, la gestion des profils utilisateurs, la publication et la consultation de messages, ainsi que les interactions sociales telles que les likes, les réponses et le suivi d'utilisateurs.

Le projet intègre également des fonctionnalités complémentaires telles que la gestion de la modération des contenus, ainsi que plusieurs extensions fonctionnelles ajoutées au-delà du périmètre initial, renforçant ainsi l'expérience utilisateur et la richesse fonctionnelle de la plateforme.

3.2 Matrice des rôles et permissions

MATRICE DES RÔLES ET PERMISSIONS – BREEZY				
FONCTIONNALITÉS	 VISITEUR	 UTILISATEUR	 MODÉRATEUR	 ADMINISTRATEUR
Fx1. Création de comptes utilisateurs	✓	✗	✗	✓
Fx2. Authentification sécurisée	✗	✓	✓	✓
Fx3. Publication de messages courts	✗	✓	✓	✓
Fx4. Affichage des messages sur le profil	✗	✓ (seulement le sien)	✓	✓
Fx5. Flux chronologique des messages	✗	✓	✓	✓
Fx6. Liker un post	✗	✓	✓	✓
Fx7. Répondre à un post sous forme de commentaire	✗	✓	✓	✓
Fx8. Répondre à un commentaire sur un post	✗	✓	✓	✓
Fx9. Suivre ou être suivi par d'autres utilisateurs	✗	✓	✓	✓
Fx10. Profil utilisateur avec informations de base	✗	✓	✓	✓
Fx11. Liste des messages publiés par l'utilisateur	✗	✓	✓	✓
Fx12. Ajout de tags aux messages	✗	✓	✓	✓
Fx13. Recherche de posts via des tags	✗	✓	✓	✓
Fx14. Notifications pour les mentions	✗	✓	✓ (modération)	✓ (administration)
Fx15. Notifications pour les likes	✗	✓	✓	✓
Fx16. Notifications pour les nouveaux followers	✗	✓	✓	✓
Fx17. Système de messages privés entre utilisateurs	✗	✓	✓	✓
Fx18. Ajout d'images aux messages	✗	✓	✓	✓
Fx19. Ajout de vidéos aux messages	✗	✓	✓	✓
Fx20. Signalement de contenu inapproprié	✗	✓	✓	✓
Fx21. Suspension ou bannissement des utilisateurs	✗	✗	✓	✓
Fx22. Interface multi-langues	✓	✓	✓	✓
Fx23. Thème personnalisé	✓	✓	✓	✓

 Autorisé
  Non autorisé
 | () Précision

3.3 Besoins non fonctionnels et priorisation MoSCoW

Les besoins non fonctionnels du projet portent principalement sur la performance, la sécurité, la maintenabilité et la qualité logicielle.

Sur le plan de la performance et de l'architecture, l'application repose sur une conteneurisation via Docker, avec des mécanismes de mise en cache intégrés afin d'optimiser certains traitements. Une architecture en microservices permet également de limiter les dépendances entre composants et d'améliorer la scalabilité globale du système.

Concernant la sécurité, l'application met en œuvre une authentification basée sur JWT, une validation stricte des données côté backend ainsi qu'une gestion rigoureuse des codes de réponse HTTP. Le déploiement inclut également une configuration HTTPS en environnement de production afin de sécuriser les échanges.

En matière de qualité logicielle, le projet intègre des tests unitaires et d'intégration, un système de linting côté backend et frontend, ainsi qu'une intégration continue (CI/CD) permettant d'automatiser les vérifications lors des déploiements.

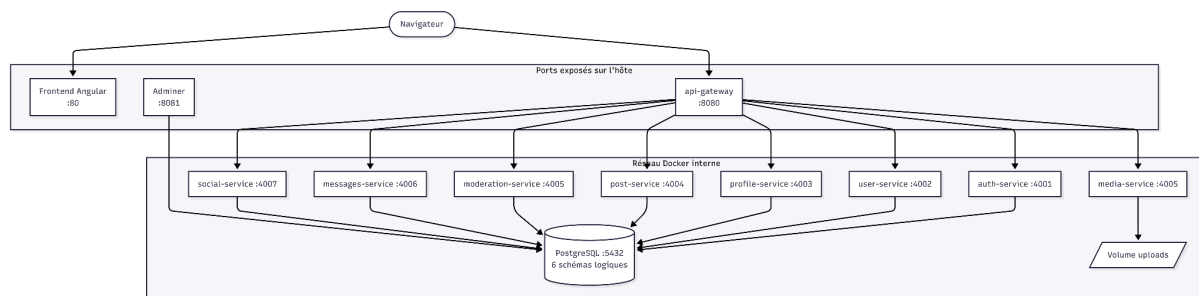
La priorisation des besoins suit une logique MoSCoW où l'ensemble des fonctionnalités prévues ont été implémentées. Les fonctionnalités essentielles (Must have) et complémentaires (Should have et Could have) ont toutes été réalisées dans le cadre du projet, sans fonctionnalité abandonnée ou reportée.

4. Architecture et conception (partie technique cœur)

Breezy est développé sous la forme d'un monorepo : l'intégralité du code source backend, frontend et configuration de déploiement réside dans un dépôt Git unique. Côté serveur, ce dépôt s'organise autour d'un seul module Go (un go.mod commun) compilé en neuf binaires indépendants, chacun s'exécutant dans son propre conteneur. Une image Docker unique, produite par un Dockerfile multi-étapes, sert l'ensemble du backend : les services ne se distinguent que par la commande de démarrage qui sélectionne le binaire à lancer. Ce choix raccourcit la durée de build et garantit surtout que tous les services partagent exactement les mêmes versions de dépendances, éliminant une source classique de bugs d'intégration.

Binaire	Port interne	Responsabilité
api-gateway	8080	Point d'entrée unique, reverse-proxy, CORS
auth-service	4001	Inscription, connexion, JWT, OAuth Google et GitHub
user-service	4002	Recherche d'utilisateurs, notifications
profile-service	4003	Profil, bio, avatar, bannière, préférences
post-service	4004	Publications, réponses, likes, reposts, sondages, signets
media-service	4005	Upload et stockage de fichiers médias
moderation-service	4005	Signalements, sanctions de comptes
messages-service	4006	Messagerie privée, conversations
social-service	4007	Abonnements, blocages, demandes de suivi

4.1 Architecture microservices : monorepo, 9 binaires, 1 module Go / 1 image Docker



4.1.1 Rôle de l'API Gateway et routage conditionnel

L'API Gateway est le seul composant exposé au réseau extérieur, sur le port 8080. Il joue le rôle de reverse-proxy : chaque requête entrante est réécrite et transmise au service interne concerné, à l'aide du reverse-proxy de la bibliothèque standard de Go (httputil), selon une logique de routage entièrement statique déclarée dans un unique `main.go`.

Deux cas de routage conditionnel méritent d'être détaillés, car ils permettent de ne jamais exposer directement les services de modération et du graphe social : le frontend dialogue toujours avec des endpoints sémantiquement cohérents, sans connaître la frontière réelle entre les services.

Le premier concerne le signalement de publications. Lorsqu'une requête POST cible un chemin se terminant par `/report`, le gateway l'achemine vers le `moderation-service`, tandis que toutes les autres requêtes sur `/api/posts/*` partent vers le `post-service`. Le frontend peut ainsi appeler `/api/posts/:id/report` comme une action naturelle du post, sans savoir qu'un service distinct traite les signalements. Le second concerne les relations sociales : les chemins contenant `/follow/`, `/follow-requests/` ou `/block/`, ainsi que ceux se terminant par `/followers` ou `/following`, sont dirigés vers le `social-service`, le reste de `/api/users/*` revenant au `user-service`.

```
// backend/cmd/api-gateway/main.go

func postRoutesProxy() gin.HandlerFunc {
    postSvc := http.StripPrefix("/api/posts",
        proxy(config.Env("POST_SERVICE_URL", "http://post-service:4004")))
    modSvc := http.StripPrefix("/api",
        proxy(config.Env("MODERATION_SERVICE_URL", "http://moderation-service:4005")))

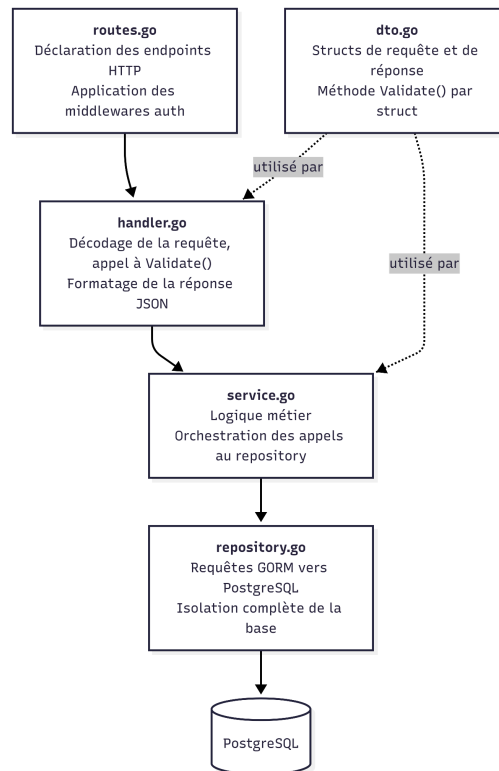
    return func(c *gin.Context) {
        if c.Request.Method == http.MethodPost &&
            strings.HasSuffix(c.Request.URL.Path, "/report") {
            modSvc.ServeHTTP(c.Writer, c.Request)
            return
        }
        postSvc.ServeHTTP(c.Writer, c.Request)
    }
}

func usersRoutesProxy() gin.HandlerFunc {
    userSvc := http.StripPrefix("/api/users",
        proxy(config.Env("USER_SERVICE_URL", "http://user-service:4002")))
    socialSvc := http.StripPrefix("/api/users",
        proxy(config.Env("SOCIAL_SERVICE_URL", "http://social-service:4007")))

    return func(c *gin.Context) {
        path := c.Request.URL.Path
        if strings.Contains(path, "/follow/") ||
            strings.Contains(path, "/follow-requests/") ||
            strings.Contains(path, "/block/") ||
            strings.HasSuffix(path, "/followers") ||
            strings.HasSuffix(path, "/following") {
            socialSvc.ServeHTTP(c.Writer, c.Request)
            return
        }
        userSvc.ServeHTTP(c.Writer, c.Request)
    }
}
```

4.1.2 Patron interne d'un service

Tous les services suivent le même découpage en quatre couches, illustré ici avec le post-service, retenu comme exemple parce qu'il concentre les fonctionnalités les plus variées.



La couche routes associe chaque chemin HTTP à son handler et lui applique le niveau d'authentification requis AuthRequired, AuthOptional, ou aucun middleware pour les routes publiques.

```
// backend/internal/post/routes.go

func RegisterRoutes(router *gin.Engine, handler *Handler) {
    router.Use(middleware.JSONBody())
    router.Use(middleware.AuthOptional())

    router.GET("/feed", handler.GetPosts)
    router.GET("/:id", handler.GetPost)
    router.GET("/tags/trending", handler.GetTrendingTags)

    authenticated := router.Group("/")
    authenticated.Use(middleware.AuthRequired())
    authenticated.POST("/", handler.CreatePost)
    authenticated.POST("/:id/like", handler.LikePost)
    authenticated.DELETE("/:id/like", handler.UnlikePost)
    authenticated.POST("/:id/repost", handler.Repost)
    authenticated.DELETE("/:id", handler.DeletePost)
}
```

La couche *DTO* porte la validation métier: chaque structure de requête expose une méthode `Validate()`, et le handler ne manipule jamais les champs bruts sans passer par elle. C'est par exemple à ce niveau qu'est imposée la limite de 280 caractères d'une publication.

```
// backend/internal/post/dto.go

type createPostRequest struct {
    Content      string          `json:"content"`
    ParentID     *uuid.UUID      `json:"parentId"`
    Visibility    string          `json:"visibility"`
    Media        []mediaRequest  `json:"media"`
    Poll         *createPollRequest `json:"poll"`
}

func (r createPostRequest) Validate() error {
    content := strings.TrimSpace(r.Content)
    if content == "" && len(r.Media) == 0 && r.Poll == nil {
        return errors.New("post content, media or poll is required")
    }
    if len([]rune(content)) > 280 {
        return errors.New("content is too long")
    }
    return nil
}
```

La couche *repository*, enfin, isole toutes les interactions avec PostgreSQL : le service ne connaît que les méthodes du repository, jamais la base directement.


```
// backend/internal/post/repository.go

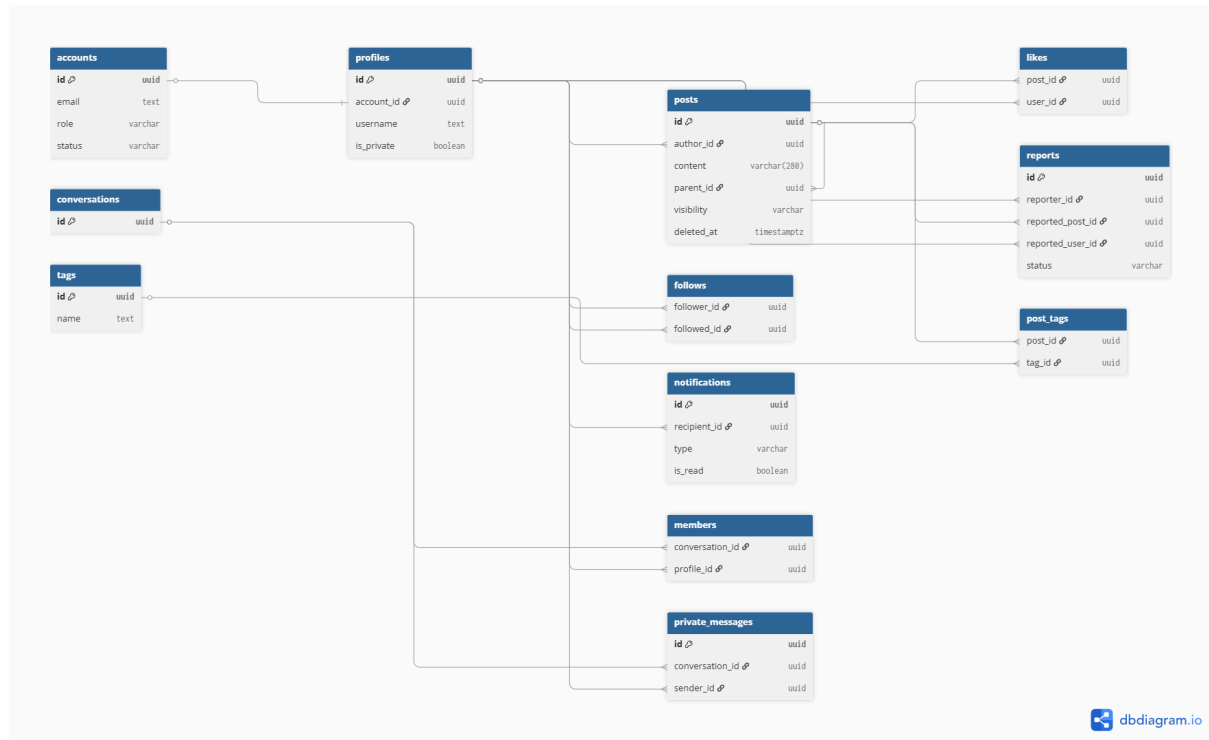
func (r *Repository) FindFeedPosts(ctx context.Context) ([]models.Post, error) {
    var posts []models.Post
    err := r.db.WithContext(ctx).
        Where("parent_id IS NULL AND deleted_at IS NULL").
        Order("created_at DESC").
        Find(&posts).Error
    return posts, err
}
```

Ce patron étant rigoureusement identique d'un service à l'autre, le code reste prévisible et la navigation entre services lors d'une revue ou d'une reprise de maintenance s'en trouve grandement facilitée.

4.2 Modèle de données : PostgreSQL à 6 schémas

Les données sont stockées dans une unique instance PostgreSQL, organisée en six schémas logiques. Chaque service n'accède qu'au schéma qui le concerne, via le `search_path` positionné dans sa variable `DATABASE_URL`. Cette isolation par schéma prévient les conflits de nommage entre services tout en conservant, contrairement à une approche multi-bases, les garanties transactionnelles ACID sur l'ensemble.

Les valeurs à cardinalité fixe sont structurées par cinq types ENUM — `user_role` (USER, MODERATOR, ADMIN), `account_status` (ACTIVE, SUSPENDED, BANNED), `post_visibility` (PUBLIC, FOLLOWERS, PRIVATE), `notification_type` et `report_status` (PENDING, REVIEWED, RESOLVED, REJECTED). Un trigger `set_updated_at()` entretient automatiquement le champ `updated_at` des tables concernées, et la suppression logique (soft delete) est mise en œuvre sur `posts` et `private_messages` au moyen d'un champ `deleted_at`, ce qui permet de masquer un contenu sans le détruire.



Le schéma DDL complet et la version DBML ayant servi à générer ce diagramme figurent en annexe B.

4.3 Conception de l'API REST et niveaux d'accès

L'API REST de Breezy expose ses routes sur trois niveaux d'accès, vérifiés côté backend dans chaque service par les middlewares AuthRequired et RequireRole.

Le niveau public n'exige aucun token : il couvre l'ensemble des routes d'authentification (inscription, connexion, vérification d'email, OAuth) ainsi que la lecture des contenus publics (fil général, profils publics, tags en tendance). Le niveau authentifié requiert un JWT valide dans l'en-tête Authorization ; il s'applique à toute action d'écriture (publier, liker, suivre, envoyer un message) comme à la lecture de contenus personnalisés (fil des abonnements, signets, notifications). Le niveau par rôle, enfin, s'appuie sur le champ role du payload JWT, contrôlé par le middleware RequireRole : les routes de modération sont réservées aux rôles MODERATOR et ADMIN, et les routes d'administration aux seuls ADMIN.

Service	Méthode	Route (depuis le gateway)	Accès
auth	POST	/api/auth/register	Public
auth	POST	/api/auth/login	Public
auth	GET	/api/auth/me	Authentifié
auth	GET	/api/auth/google	Public (OAuth)
auth	PATCH	/api/auth/accounts/:id/status	Admin
user	GET	/api/users/search	Public
social	GET	/api/users/:username/followers	Public
social	POST	/api/users/:username/follow	Authentifié
social	POST	/api/users/:username/block	Authentifié
profile	GET	/api/profiles/:username	Public
profile	PATCH	/api/profiles/me	Authentifié
post	POST	/api/posts	Authentifié
post	POST	/api/posts/:id/like	Authentifié
post	DELETE	/api/posts/:id	Authentifié (auteur)
post	GET	/api/tags/trending	Public
moderation	POST	/api/posts/:id/report	Authentifié
moderation	GET	/api/moderation/reports	Modérateur
messages	GET	/api/conversations	Authentifié
media	POST	/api/media/upload	Authentifié
notifications	GET	/api/notifications	Authentifié

4.4 Conception UX/UI : responsive mobile-first, thème clair/sombre, i18n 6 langues

Une conception responsive « mobile-first ». L'interface est construite avec Tailwind CSS en partant de la mise en page la plus contrainte une seule colonne, navigation en bas d'écran puis enrichie pour les écrans plus larges, où apparaissent une barre de navigation latérale à gauche et une colonne de droite dédiée aux suggestions et aux tendances. Ce sens de conception garantit une expérience pleinement utilisable sur téléphone, sans dégradation, ce qui correspond aux contextes à faibles ressources visés par le projet.

Un thème clair, sombre et système. Le basculement repose simplement sur l'ajout ou le retrait de la classe `.dark` sur la balise `<html>`. Toutes les couleurs de l'interface sont pilotées par des variables CSS, les design tokens centralisées en tête de `src/styles.css` : modifier une seule valeur s'y propage à tous les composants, sans toucher au TypeScript.

```
/* frontend/src/styles.css -- design tokens (extrait) */
:root {
  --c-brand: #00a9c0; /* teal principal */
  --c-text-1: #0e2a2f; /* texte principal */
  --c-text-2: #5e7a80; /* texte secondaire */
  --c-layer-1: #ffffff; /* surface card */
  --c-border: #e2edef; /* bordure standard */
  --c-danger: #f0556a; /* rouge (erreurs, likes) */
  --c-success: #22a98a; /* vert (reposts, succès) */
}
.dark {
  --c-text-1: #d8ecf2;
  --c-layer-1: rgba(8, 18, 38, 0.82);
  --c-border: rgba(28, 52, 90, 0.42);
}
```

5. Choix technologiques et écarts au sujet

5.1 Choix techniques retenus

Dans le cadre du projet Breezy, les technologies retenues ont été choisies afin de répondre à des besoins de performance, de maintenabilité et de déploiement. L'application repose sur un backend développé en Go, un frontend en Angular 21, une interface stylisée avec Tailwind CSS, ainsi qu'un environnement conteneurisé avec Docker.

5.1.1 Le choix back-end

Le choix de Go pour le backend s'explique par ses très bonnes performances d'exécution. Go est un langage compilé, ce qui permet de produire des binaires rapides, légers et directement exécutables. Il est particulièrement adapté au développement d'API web, notamment grâce à sa faible consommation mémoire et à sa capacité à gérer efficacement un grand nombre de requêtes simultanées. Dans le projet, le backend est découpé en plusieurs domaines fonctionnels, tels que l'authentification, les publications, les profils, les messages ou encore la modération. Cette organisation permet de structurer le code tout en conservant une exécution rapide et maîtrisée.

L'utilisation du framework Gin renforce cette approche. Gin est un framework HTTP léger et performant, qui permet de créer des routes, des middlewares et des réponses JSON sans ajouter de surcharge excessive. Il est donc adapté à une API REST comme celle du projet Breezy, où les échanges entre le frontend et le backend sont fréquents.

5.1.2 Le choix front-end

Pour le frontend, le choix d'Angular 21 permet de construire une application robuste et structurée. Angular offre une architecture basée sur les composants, les services, les guards et les routes, ce qui facilite l'organisation d'une application riche en fonctionnalités. Dans le projet, les différentes pages comme le fil d'actualité, les profils, la messagerie, les notifications, la modération ou l'administration sont séparées proprement, ce qui améliore la lisibilité et la maintenabilité du code.

Angular apporte également des optimisations importantes en matière de performance. Le projet utilise notamment le lazy loading, qui permet de charger certaines parties de l'application uniquement lorsqu'elles sont nécessaires. Cela réduit la taille initiale du chargement côté navigateur et améliore l'expérience utilisateur. De plus, la configuration Angular du projet intègre le SSR, c'est-à-dire le rendu côté serveur, ce qui peut améliorer le temps d'affichage initial et rendre l'application plus réactive au premier chargement.

Le choix de Tailwind CSS permet de créer rapidement une interface claire et cohérente. Il fonctionne avec des classes déjà prêtes, directement utilisées dans le code HTML, ce qui évite d'écrire beaucoup de CSS à la main. Cela rend le développement plus simple et plus

rapide. Dans le projet Breezy, Tailwind aide aussi à garder un style homogène sur toutes les pages, notamment pour les couleurs, les boutons, les cartes, les espacements et le mode sombre.

5.1.3 Le choix du déploiement

Enfin, Docker a été choisi afin de simplifier le déploiement et l'exécution du projet. Grâce à Docker Compose, les différents services de l'application peuvent être lancés dans des conteneurs isolés : backend, frontend, base de données PostgreSQL et outils associés. Cette conteneurisation garantit que l'application fonctionne de la même manière sur différents environnements, que ce soit en développement ou en production. Elle limite les problèmes liés aux différences de configuration entre les machines des développeurs.

Docker facilite également la séparation des responsabilités entre les services. Chaque composant peut disposer de son propre environnement d'exécution, de ses dépendances et de sa configuration. Dans le cas de Breezy, cette approche est cohérente avec l'architecture du projet, qui distingue clairement le frontend, le backend et la base de données. Elle améliore donc la portabilité, la reproductibilité et la facilité de maintenance de l'application.

5.2 Justification des écarts avec le sujet initial

Le choix de Go plutôt qu'Express.js s'explique par le fait que Go apporte davantage de performance et de robustesse pour le backend. Contrairement à Express, qui repose sur JavaScript et Node.js, Go est un langage compilé : le serveur est donc plus rapide à l'exécution et consomme généralement moins de ressources. Go est aussi très efficace pour gérer plusieurs requêtes en parallèle grâce aux goroutines, ce qui est un avantage pour une application comme Breezy où les utilisateurs peuvent interagir en même temps avec les posts, les messages, les likes ou les notifications. Enfin, Go apporte un typage fort et une compilation stricte, ce qui permet de détecter plus d'erreurs avant le lancement de l'application qu'avec Express.

Le choix d'Angular plutôt que React s'explique par le fait qu'Angular offre un cadre plus complet dès le départ. Contrairement à React, qui est principalement une bibliothèque d'interface et qui nécessite souvent d'ajouter plusieurs outils externes, Angular intègre déjà le routage, les formulaires, les services, les guards, les intercepteurs HTTP et une architecture claire basée sur les composants. Pour un projet comme Breezy, avec de nombreuses pages et fonctionnalités, Angular permet donc de garder une structure plus homogène et plus facile à maintenir. Angular impose aussi davantage de conventions, ce qui aide l'équipe à développer de manière cohérente sur l'ensemble du projet.

6. Réalisation / Développement

6.1 Vue D'ensemble

L'application repose sur une architecture séparant clairement le frontend Angular et le backend Go. Le frontend constitue la couche d'interface utilisateur : il gère les pages, les composants, les formulaires, les appels HTTP, l'état utilisateur et l'expérience de navigation. Le backend est découpé en services spécialisés, exposés au frontend via une API Gateway unique. Cette organisation permet au frontend de n'appeler qu'un seul point d'entrée /api, tandis que la gateway redirige ensuite les requêtes vers le service concerné.

Élément	Rôle technique
api-gateway	Point d'entrée backend unique, reverse proxy vers les services internes
auth-service	Connexion, inscription, sessions, cookies, OAuth, rôles
user-service	Recherche utilisateurs, suggestions, administration des comptes
profile-service	Profil utilisateur, bio, avatar, bannière, confidentialité
post-service	Posts, réponses, likes, reposts, bookmarks, sondages, tags
media-service	Upload et exposition des médias
moderation-service	Signalements, sanctions, bannissements, suspensions
messages-service	Conversations privées, messages, réactions, préférences
social-service	Follow, unfollow, demandes d'abonnement, blocage
Front Angular	Pages, composants, services HTTP, guards, thème, mode invité

La structure frontend est organisée autour de pages fonctionnelles (home, feed, profile, messages, notifications, moderation, administration) et de services partagés dans core/services. Les appels backend passent par HttpClient et utilisent une URL de base configurable selon l'environnement : http://localhost:8080/api en développement et /api en production.

6.2 Fonctionnalités Obligatoires

Les fonctionnalités demandées dans le sujet ont été intégrées dans le frontend et reliées aux services backend correspondants. Le frontend déclenche les actions depuis les composants Angular, tandis que la logique métier est assurée côté backend par les services Go.

Fonctionnalité	Implémentation technique
Compte utilisateur	Inscription, connexion, déconnexion, session par cookies HTTP-only
Post limité à 280 caractères	Validation frontend et backend sur le contenu des publications
Fil d'actualité	Récupération des posts via /api/posts/feed et affichage dans Angular
Likes	Routes POST /posts/:id/like et DELETE /posts/:id/like
Réponses	Champ parent_id sur les posts, permettant de rattacher une réponse à un post parent
Profil	Service dédié aux profils avec modification des informations utilisateur
Notifications	Notifications de likes, réponses, mentions et modération
Recherche	Recherche d'utilisateurs, de posts et de tags
Thème	Gestion du thème côté frontend avec service Angular dédié
Mode invité	Navigation limitée avec blocage des actions nécessitant un compte

L'authentification est sécurisée par des cookies HTTP-only. Le frontend ne stocke pas les JWT dans le localStorage ; il conserve seulement les informations utiles à l'affichage de la session. Les routes Angular sont protégées par des guards afin de différencier les utilisateurs connectés, les invités et les rôles spécifiques comme ADMIN ou MODERATOR.

6.3 Fonctionnalités Au-delà Du Sujet

Plusieurs fonctionnalités supplémentaires ont été développées afin d'enrichir l'application et de la rapprocher d'un réseau social complet.

La messagerie privée permet aux utilisateurs de créer des conversations, d'envoyer des messages, de modifier ou supprimer leurs messages et d'utiliser des réactions. Les sondages peuvent être attachés à un post avec plusieurs options, une durée éventuelle et un vote unique par utilisateur. Les reposts permettent de republier un contenu dans le fil, tandis que les bookmarks permettent de sauvegarder des publications.

Les profils privés ajoutent une couche de confidentialité : un profil peut limiter la visibilité de ses contenus à ses abonnés. Le blocage empêche certaines interactions et masque les contenus concernés. L'authentification OAuth a également été intégrée pour permettre la connexion via des fournisseurs externes. La modération permet aux rôles autorisés de traiter les signalements, suspendre ou bannir un utilisateur. Enfin, les mentions et la traduction améliorent l'expérience utilisateur autour des contenus textuels.

6.4 Implémentations Phares

6.4.1 Réponses Imbriquées Et Sondages

Les réponses sont implémentées directement dans la table des posts grâce au champ `parent_id`. Un post classique possède un `parent_id` nul, tandis qu'une réponse contient l'identifiant du post auquel elle répond. Cette approche évite de créer une table séparée pour les commentaires et permet de réutiliser la même logique métier pour les posts et les réponses.

Côté backend, la récupération des réponses repose sur une recherche récursive applicative : le service charge les réponses directes d'un post, puis recherche les réponses de ces réponses afin de reconstruire un arbre de discussion. Côté frontend, la page de détail d'un post recompose cet arbre et l'affiche sous forme de réponses imbriquées.

Les sondages sont rattachés à un post. Un sondage contient plusieurs options, un compteur de votes par option, une date éventuelle d'expiration et une date de fermeture manuelle. Le vote unique est garanti par le modèle `PollVote`, qui associe un utilisateur à un sondage avec un index unique sur le couple (`poll_id`, `user_id`). Le backend vérifie également qu'un utilisateur n'a pas déjà voté avant d'enregistrer un nouveau vote.

Lorsqu'un vote est accepté, deux actions sont réalisées dans une transaction : création du vote dans `poll_votes`, puis incrément du compteur de l'option choisie. Cela permet de conserver un affichage performant des résultats, sans recompter tous les votes à chaque lecture.

6.4.2 Sanctions Réversibles

Le système de modération ne supprime pas définitivement les données d'un utilisateur sanctionné. Lorsqu'un compte est suspendu ou banni, le profil est anonymisé et ses posts sont masqués en passant leur visibilité à PRIVATE. Les valeurs originales sont conservées dans des colonnes dédiées afin de pouvoir restaurer l'état initial si la sanction est levée.

Cette logique est centralisée dans le package sanction. Elle est utilisée par le service de modération lors d'un bannissement ou d'une suspension, mais aussi par le service d'authentification lorsqu'une suspension expire. L'ensemble est exécuté dans une transaction afin de garantir la cohérence entre le statut du compte, l'anonymisation du profil et le masquage des publications.

6.4.3 Chiffrement de bout en bout des messages privés

La messagerie privée de Breezy repose sur un chiffrement de bout en bout : le serveur stocke uniquement un ciphertext opaque et n'a à aucun moment accès au contenu des échanges entre utilisateurs. Cette architecture garantit la confidentialité des messages même en cas de compromission de la base de données.

Génération et publication des clés

Lors de son premier accès à la messagerie, le client Angular génère une paire de clés X25519 (courbe elliptique Diffie-Hellman, clés de 32 octets). La clé privée ne quitte jamais le navigateur de l'utilisateur : elle est conservée dans l'IndexedDB local et n'est transmise à aucun service. La clé publique est en revanche publiée sur le profil de l'utilisateur via l'endpoint PATCH /api/profiles/me. Le backend valide systématiquement qu'elle constitue bien une clé X25519 valide de 32 octets encodée en base64 avant de l'enregistrer.

```
// backend/internal/profile/dto.go
func (r updateProfileRequest) Validate() error {
    if r.PublicKey != nil {
        decoded, err :=
base64.StdEncoding.DecodeString(strings.TrimSpace(*r.PublicKey))
        if err != nil || len(decoded) != 32 {
            return errors.New("publicKey must be a base64-encoded
32-byte X25519 key")
        }
    }
    return nil
}
```

Chiffrement côté client

Avant l'envoi d'un message, le client récupère la clé publique du destinataire exposée dans la réponse de l'endpoint de conversation. Il chiffre ensuite le contenu avec l'algorithme NaCl box (Curve25519 + XSalsa20 + Poly1305), qui combine échange de clé Diffie-Hellman et chiffrement authentifié en une seule opération. Le ciphertext et le nonce produits sont transmis au backend dans le corps de la requête.

Stockage côté serveur

Le serveur ne déchiffre jamais les messages. Il enregistre le ciphertext base64 dans le champ Content et le nonce dans le champ Nonce de la table private_messages, sans aucune interprétation du contenu.

```
// backend/internal/models/messages.go
type PrivateMessage struct {
    // Content contient le ciphertext base64 chiffré côté client (NaCl
    // box).
    // Le serveur le stocke de manière opaque sans jamais le déchiffrer.
    Content string
    // Nonce fourni par le client, nécessaire au déchiffrement côté
    // destinataire.
    Nonce string
}

// backend/internal/messages/service.go
// Le serveur persiste le ciphertext tel quel, sans accès au texte
// clair.
message := models.PrivateMessage{
    ConversationID: conversationID,
    SenderID:      current.ID,
    Content:       strings.TrimSpace(content), // ciphertext (opaque)
    Nonce:         strings.TrimSpace(nonce),
}
```

Compatibilité rétroactive

Les messages envoyés avant la mise en place du chiffrement possèdent un champ Nonce vide. Le client les détecte et les affiche en texte clair sans tentative de déchiffrement, garantissant ainsi qu'aucun historique existant n'est perdu lors de la migration.

7. Sécurité

7.1 Authentification

L'authentification de Breezy repose sur un schéma JWT, sans bibliothèque tierce comme `golang-jwt` : l'émission et la vérification du token sont implémentées directement dans `backend/internal/security/access_token.go`. Le jeton d'accès contient trois informations seulement : l'identifiant du compte (`sub`), son rôle (`role`) et sa date d'expiration (`exp`), signées en HMAC-SHA256 avec un secret partagé lu depuis la variable d'environnement `JWT_SECRET` (le service refuse de démarrer si elle est absente).

```
// backend/internal/security/access_token.go
func SignAccessToken(accountID uuid.UUID, role string, ttl
time.Duration) (string, error) {
    claims := AccessClaims{
        Subject:  accountID.String(),
        Role:     role,
        ExpiresAt: time.Now().UTC().Add(ttl).Unix(),
    }
    header := map[string]string{"alg": "HS256", "typ": "JWT"}
    return signJWT(header, claims)
}
```

La durée de vie du jeton d'accès est volontairement très courte (5 minutes), ce qui limite la fenêtre d'exploitation en cas de vol du token. Pour éviter de réauthentifier l'utilisateur toutes les 5 minutes, un jeton de rafraîchissement valable 30 jours est délivré en parallèle. Ce refresh token n'est jamais stocké en clair côté serveur : seule son empreinte SHA-256 est conservée dans la table sessions, via la fonction `hashToken()`. Ainsi, une fuite de la base de données ne permet pas de rejouer les sessions actives.

```
// backend/internal/auth/service.go
const (
    accessTokenTTL = 5 * time.Minute // fenêtre d'exploitation minimisée
    refreshTokenTTL = 30 * 24 * time.Hour // sessions longues via refresh
)
```

Le rafraîchissement est en outre rotatif : à chaque appel à `/api/auth/refresh`, le service vérifie la session associée au hash reçu, puis révoque immédiatement cette session (`RevokeSession`) avant d'en créer une nouvelle avec un jeton inédit. Un ancien refresh token ne peut donc jamais être réutilisé deux fois, c'est une protection classique contre le vol de jeton par interception.

7.2 Autorisation RBAC, bcrypt et politique de mot de passe forte

Le contrôle d'accès par rôle repose sur trois middlewares Gin définis dans backend/internal/middleware/auth_required.go : AuthRequired() bloque la requête (401) si aucun token valide n'est présent (en-tête Authorization: Bearer ... ou cookie), AuthOptional() enrichit le contexte sans bloquer les routes publiques, et RequireRole() compare le rôle porté par le JWT à un rôle minimal requis grâce à une fonction de classement :

```
// backend/internal/middleware/auth_required.go
func roleRank(role string) int {
    switch strings.ToUpper(role) {
    case RoleAdmin:    return 3
    case RoleModerator: return 2
    case RoleUser:     return 1
    default:           return 0
    }
}

func RequireRole(requiredRole string) gin.HandlerFunc {
    return func(c *gin.Context) {
        role, ok := Role(c)
        if !ok || roleRank(role) < roleRank(requiredRole) {
            c.AbortWithStatusJSON(http.StatusForbidden, gin.H{"error": "insufficient
role"})
            return
        }
        c.Next()
    }
}
```

Cette hiérarchie (ADMIN > MODERATOR > USER) est appliquée indépendamment dans chaque service consommant le JWT, conformément au principe « chaque service vérifie le token lui-même » : il n'existe pas de point de contrôle d'autorisation centralisé que la compromission d'un seul service suffirait à contourner.

Les mots de passe ne sont jamais stockés en clair : ils sont hachés avec bcrypt.GenerateFromPassword au coût par défaut (bcrypt.DefaultCost, soit un facteur de coût 10), et comparés à la connexion via bcrypt.CompareHashAndPassword, qui s'exécute en temps constant et résiste aux attaques par timing. La politique de mot de passe, centralisée dans backend/internal/validation/validation.go, impose à l'inscription un mot de passe d'au moins 12 caractères contenant une majuscule, un chiffre et un caractère spécial, et rejette explicitement tout mot de passe qui contiendrait le nom d'utilisateur, la partie locale de l'e-mail ou un mot significatif du nom affiché, c'est une protection simple mais efficace contre les mots de passe triviaux dérivés de l'identité du compte.

7.3 Validation, CORS, injections

La validation des entrées combine deux niveaux : les tags `binding:"required,email,min=..."` de Gin pour les contraintes de forme, et une méthode `Validate()` propre à chaque DTO pour la logique métier (limite des 280 caractères d'un post, cohérence d'un mot de passe avec l'identité du compte, etc.), comme décrit en section 4.1.2. Aucune requête n'atteint la couche service sans être passée par cette double validation.

Le CORS est géré par un middleware dédié au niveau de l'API Gateway (`backend/internal/middleware/cors.go`), paramétré par la variable d'environnement `CORS_ORIGIN`. Une seule origine est autorisée à la fois (celle du frontend, en développement comme en production), plutôt qu'un wildcard *, ce qui est nécessaire pour autoriser l'envoi de cookies (`Allow-Credentials: true`).

```
// backend/internal/middleware/cors.go
func CORS(next http.Handler) http.Handler {
    origin := config.Env("CORS_ORIGIN", "http://localhost:4200")
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request)
    {
        w.Header().Set("Access-Control-Allow-Origin", origin)
        w.Header().Set("Access-Control-Allow-Methods", "GET, POST, PUT,
PATCH, DELETE, OPTIONS")
        w.Header().Set("Access-Control-Allow-Headers", "Content-Type,
Authorization")
        w.Header().Set("Access-Control-Allow-Credentials", "true")
    })
}
```

Un middleware de limitation de débit est appliqué au niveau de l'API Gateway afin de protéger l'application contre les abus, les attaques par force brute et l'énumération de comptes. L'algorithme retenu est le seau à jetons (Token Bucket) : chaque adresse IP dispose d'un seau dont les jetons se régénèrent à débit constant. Chaque requête consomme un jeton ; toute requête arrivant à seau vide reçoit une réponse HTTP 429 Too Many Requests. Un processus de nettoyage s'exécute toutes les 10 minutes afin de purger les entrées inactives et d'éviter toute fuite mémoire.

```
// backend/internal/middleware/rate_limit.go
func NewRateLimiter(maxRequests int, window time.Duration) *RateLimiter
{
    rl := &RateLimiter{
        visitors: make(map[string]*visitor),
        rate:      float64(maxRequests) / window.Seconds(),
        burst:     float64(maxRequests),
    }
}
```

```

    }
    go rl.cleanupLoop()
    return rl
}

func (rl *RateLimiter) Middleware() gin.HandlerFunc {
    return func(c *gin.Context) {
        if !rl.allow(clientIP(c.Request)) {
            c.AbortWithStatusJSON(http.StatusTooManyRequests,
                gin.H{"error": "too many requests, please retry later"})
            return
        }
        c.Next()
    }
}

```

Deux niveaux de limitation sont configurés dans le gateway : une limite générale de 100 requêtes par minute s'applique à l'ensemble des routes de l'API, et une limite stricte de 10 requêtes par minute est imposée sur les routes `/api/auth/*`, qui constituent la surface d'attaque la plus exposée.

```

// backend/cmd/api-gateway/main.go
// Limite générale sur l'ensemble de l'API
router.Use(middleware.NewRateLimiter(100, time.Minute).Middleware())

// Limite renforcée sur les routes d'authentification sensibles aux abus
authRateLimiter := middleware.NewRateLimiter(10,
time.Minute).Middleware()
router.Any("/api/auth/*path", authRateLimiter, gin.WrapH(
    http.StripPrefix("/api/auth", proxy(config.Env("AUTH_SERVICE_URL",
        "http://auth-service:4001")))),
))

```

L'état est maintenu en mémoire par IP, ce qui est suffisant pour un déploiement mono-instance. Une implémentation Redis serait nécessaire pour passer à un environnement multi-instance, et constitue un axe d'amélioration identifié (section 9.3).

Côté base de données, l'usage systématique de GORM, y compris pour les requêtes SQL brutes (`Raw()`) utilisées pour les jointures complexes ou la recherche repose exclusivement sur des paramètres positionnels (?), jamais sur de la concaténation de chaînes. Aucune requête construite par interpolation de variables utilisateur n'a été identifiée dans le code, ce qui écarte le risque d'injection SQL classique.

Une limite est assumée et non corrigée dans le cadre de ce livrable : l'absence d'en-tête Content-Security-Policy côté Nginx ou côté backend, qui aurait renforcé la défense en profondeur contre les attaques XSS en plus de la protection déjà apportée par les cookies HttpOnly.

7.4 Tests unitaires et couverture de code

La suite de tests du backend Breezy s'appuie exclusivement sur la bibliothèque standard Go (testing, net/http/httptest) sans framework de test externe. Trois niveaux de tests ont été mis en place.

Tests unitaires purs. Ils vérifient la logique métier de façon totalement isolée, sans I/O ni réseau. Les tests sont écrits en style table-driven — idiome Go qui consiste à regrouper l'ensemble des cas d'un même comportement dans une slice de structs anonymes — ce qui permet de couvrir en quelques lignes toutes les combinaisons d'entrées et de résultats attendus. Ce style est utilisé notamment pour les règles de visibilité des posts, la validation des DTOs, et la logique de sanction. Le package internal/security teste intégralement la signature et la vérification des JWT HS256 : token valide, token expiré, signature falsifiée, encodage base64 corrompu, payload JSON invalide, et panique en cas d'absence de JWT_SECRET — chaque branche d'erreur est exercée indépendamment.

Tests avec base de données mockée. La bibliothèque go-sqlmock (DATA-DOG) est utilisée pour simuler les interactions GORM / PostgreSQL en scriptant précisément les requêtes SQL attendues et leurs réponses. Cela permet de couvrir les branches d'erreur des handlers et repositories — base indisponible, enregistrement introuvable, contrainte d'unicité violée — sans démarrer de vrai serveur PostgreSQL. Le mode dry-run de GORM est également exploité pour exercer l'ensemble des méthodes de service sans persistance réelle. Les tests en mode base indisponible (handler avec nil injecté à la place du *gorm.DB) vérifient que chaque endpoint répond HTTP 503 de manière cohérente plutôt que de paniquer.

Tests d'intégration avec Testcontainers. Le package internal/integration démarre un conteneur PostgreSQL réel via testcontainers-go, applique les migrations SQL complètes, injecte des fixtures, puis exerce l'ensemble des services de manière bout-en-bout : inscription, vérification d'e-mail, authentification, rotation de refresh token, réinitialisation de mot de passe, OAuth (Google / GitHub), publication de posts, follow, likes, messagerie, modération et notifications. Ces tests sont marqués t.Skip() lorsque le flag -short est actif, afin de ne pas alourdir la pipeline CI qui ne dispose pas de Docker.

La couverture toutes instructions confondues atteint 95,2 % sur l'ensemble du backend, mesurée par microservice.

Breezy backend coverage

Genere le 2026-06-24 19:10:06. Le rapport all couvre tout le backend, les autres lignes isolent chaque service.

Service	Coverage	Rapport visuel	Detail fonctions	Profil brut
all	95.2%	HTML	Fonctions	coverage/all.out
api-gateway	91.4%	HTML	Fonctions	coverage/api-gateway.out
auth-service	93.0%	HTML	Fonctions	coverage/auth-service.out
user-service	90.0%	HTML	Fonctions	coverage/user-service.out
profile-service	89.6%	HTML	Fonctions	coverage/profile-service.out
post-service	92.0%	HTML	Fonctions	coverage/post-service.out
media-service	79.5%	HTML	Fonctions	coverage/media-service.out
moderation-service	89.6%	HTML	Fonctions	coverage/moderation-service.out
messages-service	93.3%	HTML	Fonctions	coverage/messages-service.out
social-service	87.4%	HTML	Fonctions	coverage/social-service.out

8. Qualité, DevOps et déploiement

8.1 Qualité et tests : stratégie, lint, revue de code (CODEOWNERS) et dette technique assumée

Dans le cadre d'un POC mené en deux semaines, nous avons fait un choix assumé : tester la logique critique plutôt que viser une couverture exhaustive. Les tests se concentrent sur les zones où une régression aurait un impact direct sur la sécurité ou le métier, et sont écrits sous forme de *table-driven tests* (t.Run), l'idiome standard de Go qui regroupe plusieurs cas dans une même fonction. Cinq suites unitaires couvrent le backend (go test ./...). Côté frontend, un *smoke test* (app.spec.ts) valide l'amorçage de l'application via Karma/Jasmine (ng test --watch=false).

Fichier de test	Ce qui est vérifié	Pourquoi c'est critique
middleware/role_test.go	RBAC : rôle suffisant accepté, rôle insuffisant rejeté	Cœur du contrôle d'accès (§7.2)
middleware/json_body_test.go	Rejet d'un corps non-JSON ou trop volumineux, acceptation d'un corps vide	Validation des entrées / surface d'attaque
post/service_test.go	CanViewPost (visibilité selon rôle/auteur) et NormalizePostVisibility	Règles de confidentialité des posts
server/router_test.go	404, 405 et récupération des panics, toujours en JSON	Robustesse et cohérence du contrat d'API
media/handler_test.go	Upload d'un PNG valide	Chemin d'upload des médias

Linters

La qualité du code est garantie de façon automatisée, à l'identique en local et en intégration continue. Côté Go, trois niveaux se superposent : gofmt pour le formatage, go vet pour les erreurs sémantiques de la bibliothèque standard, et golangci-lint (v2.12), piloté par backend/.golangci.yml. La configuration part de default: none puis active explicitement une vingtaine de linters orientés correction et sécurité parmi lesquels errcheck en mode strict (aucune erreur ignorée silencieusement), bodyclose, sqlclosecheck et rowserrcheck (fuites de ressources HTTP/SQL), errorlint, forcetypeassert, staticcheck ou ineffassign. L'option new: true cible les problèmes introduits par une pull request sans bloquer sur la dette préexistante. Côté Angular, ESLint (ng lint) et Prettier (printWidth 100, *single quotes*) assurent un style homogène entre contributeurs.

Revue de code (CODEOWNERS)

Le projet suit Gitflow (main ← develop ← feature/*) : chaque fonctionnalité passe par une pull request et au moins une revue obligatoire avant le merge. Le fichier .github/CODEOWNERS automatise l'affectation des relecteurs par périmètre le code backend (Go, migrations, Dockerfile) à un membre, le frontend (Angular, package.json, angular.json) à un autre, et la configuration partagée (.github/, documentation) aux deux. GitHub désigne ainsi automatiquement le bon relecteur selon les fichiers modifiés, ce qui responsabilise chacun sur son domaine tout en garantissant un regard croisé sur les fichiers communs. *(Reprends les pseudos/noms exacts de ton .github/CODEOWNERS cf. le check plus haut, le rapport nomme de vraies personnes.)*

Dette technique assumée

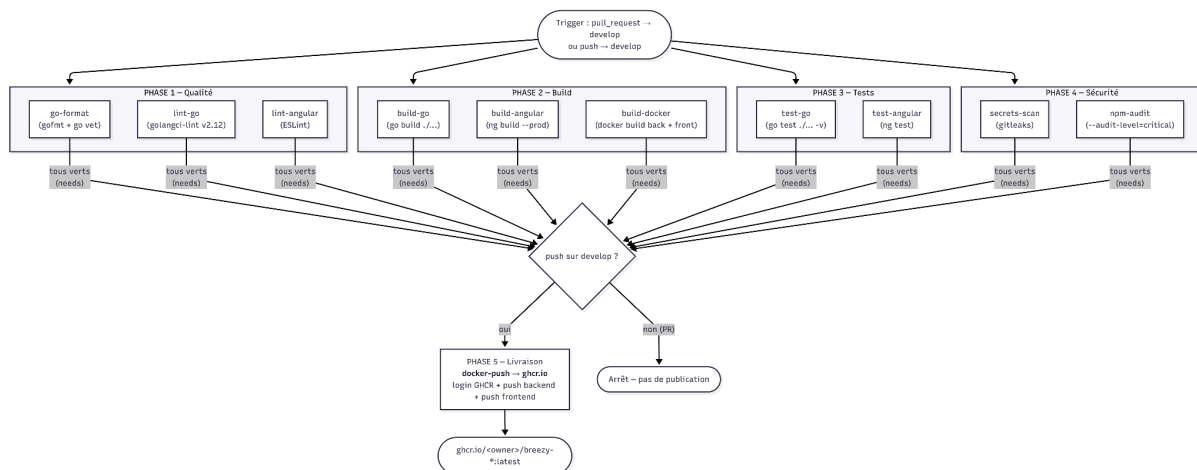
Fidèles à la priorité « une application qui tourne et que l'on sait défendre » plutôt qu'à la perfection du code, nous assumons explicitement plusieurs limites, documentées et tracées dans le backlog : une couverture de tests partielle la logique critique est testée, mais ni l'exhaustivité des handlers ni les parcours bout-en-bout (pas de tests e2e) ; une intégration backend incomplète sur le user-service (graphe de follow, suggestions, recherche), pour lequel le front s'appuie encore sur des données simulées ; l'absence de temps réel sur les notifications, rafraîchies au chargement ; et une sécurité volontairement bornée, sans *rate limiting* ni CSP. Cette dette relève d'un choix de priorisation, non d'un oubli.

8.2 CI/CD GitHub Actions et conteneurisation Docker multi-stage

Pipeline d'intégration continue

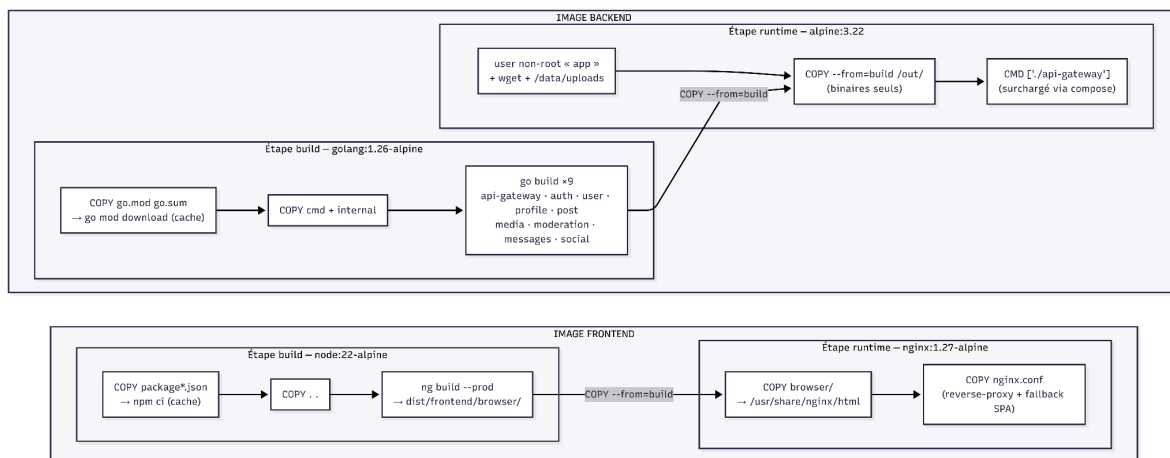
Le workflow `.github/workflows/ci-cd.yml` se déclenche sur chaque pull request vers develop et sur chaque push sur develop. Il s'organise en cinq phases regroupant onze jobs (voir diagramme 8.2). Les jobs d'une même phase s'exécutent en parallèle, et la mise en cache des dépendances (go.sum via setup-go, npm via setup-node) accélère les exécutions répétées.

Les quatre premières phases qualité, build, tests et sécurité totalisent dix jobs qui valident le code sans rien publier : formatage et vet, golangci-lint et ESLint, compilation des neuf services Go et build de production Angular, construction des images Docker, exécution des tests Go et Angular, puis analyse de sécurité (détection de secrets avec gitleaks et npm audit sur les dépendances critiques). La cinquième phase, la livraison, est conditionnelle : le job `docker-push` ne s'exécute (needs sur les dix jobs précédents, if sur un push vers develop) qu'après un merge validé et uniquement si toute la chaîne est verte. Une pull request ne publie donc jamais d'image elle ne fait que valider. Lorsque la condition est remplie, les images `breezy-backend` et `breezy-frontend` sont poussées sur le GitHub Container Registry (ghcr.io).



Conteneurisation Docker multi-étapes

Les deux images reposent sur le patron du *build multi-étapes* (voir diagramme) : une étape de compilation lourde, suivie d'une image finale minimale ne contenant que l'artefact exécutable. Le gain est triple des images finales légères, une surface d'attaque réduite (ni chaîne d'outils Go ni Node.js dans le runtime) et un cache de build efficace, les manifestes de dépendances étant copiés avant le code source, de sorte que les dépendances ne sont retéléchargées que lorsqu'elles changent. Le backend mérite une mention particulière : un monorepo, un seul module Go et une seule image, mais neuf binaires compilés en une passe, chaque service ne se distinguant ensuite que par sa commande de démarrage. Le frontend suit le même principe, l'image finale nginx ne conservant que le build statique et sa configuration de reverse-proxy.



8.3 Déploiement en production : VPS, nginx, HTTPS Let's Encrypt, seed de démo

8.3.1 Hébergement sur serveur privé virtuel (VPS)

Breezy est hébergée sur un VPS OVH (modèle VPS-1, 2 vCœurs, 4 Go de RAM, 40 Go de stockage SSD) sous Ubuntu 26.04 LTS. Ce choix répond à deux contraintes : disposer d'une adresse IP publique fixe nécessaire au nom de domaine et au certificat TLS tout en restant dans le budget d'un projet étudiant. Toute la pile s'exécute via Docker et Docker Compose ; seuls Docker Engine et Certbot sont installés sur l'hôte, ce qui rend l'environnement de production identique à celui du développement local. Le déploiement est manuel, par SSH : clonage du dépôt, configuration du fichier `.env` de production, puis reconstruction et relance des conteneurs.

8.3.2 Serveur web et reverse proxy (Nginx)

Nginx s'exécute à l'intérieur du conteneur frontend seul conteneur à exposer des ports publics (80 et 443) produit par un build multi-étapes : une image `node:22-alpine` compile l'application Angular, puis une image `nginx:1.27-alpine` n'en conserve que le build client. Bien que le projet soit configuré en mode SSR, seul ce build statique est déployé (sans processus Node) : l'application fonctionne donc en rendu côté client, Nginx renvoyant le fichier de repli `index.csr.html` pour toute URL ne correspondant pas à un fichier physique, et déléguant ainsi le routage au routeur Angular. Nginx joue également le rôle de reverse proxy : toute requête préfixée par `/api/` est transmise à l'api-gateway sur le réseau Docker interne, avec propagation des en-têtes `Host`, `X-Real-IP` et `X-Forwarded-Proto`. La résolution des noms de services est confiée au résolveur DNS interne de Docker, et aucun des microservices backend n'est accessible depuis l'extérieur du réseau.

8.3.3 Chiffrement et certificats (HTTPS / Let's Encrypt)

Le trafic est chiffré de bout en bout par un certificat TLS Let's Encrypt (clé ECDSA), émis gratuitement via Certbot installé sur l'hôte. L'émission s'appuie sur la méthode standalone : Certbot ouvre temporairement son propre serveur sur le port 80 pour répondre au défi ACME HTTP-01, le conteneur frontend étant arrêté le temps de l'opération. Les certificats restent sur l'hôte et sont montés en lecture seule dans le conteneur frontend (`/etc/letsencrypt:/etc/letsencrypt:ro`) : la clé privée n'est ainsi jamais copiée dans l'image Docker. Tout le trafic HTTP est redirigé en permanence vers HTTPS (code 301), et seuls les protocoles TLS 1.2 et 1.3 sont autorisés, avec des suites de chiffrement robustes, conformément aux recommandations de l'ANSSI.

Le renouvellement est automatisé par le timer `systemd certbot.timer`, qui vérifie deux fois par jour et renouvelle le certificat à moins de trente jours de son expiration. La méthode standalone nécessitant le port 80, des hooks d'arrêt et de relance du conteneur frontend encadrent l'opération ; leur bon fonctionnement a été validé via `certbot renew --dry-run`.

8.3.4 Initialisation des données de démonstration

Deux mécanismes peuplent la base de démonstration. Le premier, automatique, est intégré à auth-service : la fonction `SeedDemoAccounts` s'exécute à chaque démarrage lorsque la variable `SEED_DEMO_ACCOUNTS` est active, et crée quinze comptes couvrant les trois rôles applicatifs (ADMIN, MODERATOR, USER), de manière idempotente et avec des mots de passe hachés par `bcrypt`. Le second, le script `backend/scripts/seed_demo_full.sql` exécuté une seule fois après le premier démarrage, ajoute un jeu de données social plus riche posts, réponses, relations de suivi, likes, notifications et tags dans une transaction atomique dont les clauses `ON CONFLICT` garantissent qu'il reste ré-exécutable sans doublon.

9. Bilan et perspectives

9.1 Résultats au regard des objectifs et compétences mobilisées

Le projet Breezy a permis de livrer une application de microblogging fonctionnelle et sécurisée, respectant l'intégralité du cahier des charges initial tout en proposant des fonctionnalités avancées. Sur le plan technique, l'architecture en microservices (9 services) orchestrée par une API Gateway et conteneurisée via Docker a démontré sa pertinence pour garantir l'évolutivité et la maintenabilité de la solution. L'équipe a su mobiliser des compétences variées en développement Go (backend), Angular (frontend), gestion de base de données PostgreSQL et mise en œuvre de bonnes pratiques DevOps (CI/CD GitHub Actions, HTTPS).

9.2 Difficultés rencontrées et compromis assumés

Le principal défi a été le respect du délai contraint d'environ trois semaines face à la complexité d'une architecture distribuée. Pour pallier ce risque, une priorisation rigoureuse a été appliquée, et une organisation itérative a été privilégiée. Des choix de conception ont été faits pour optimiser le temps de développement, comme l'utilisation d'un monorepo pour le backend afin d'éliminer les bugs d'intégration liés aux versions de dépendances. Concernant la sécurité, bien que des standards robustes soient en place (JWT, bcrypt, RBAC), certaines limites ont été assumées volontairement, comme l'absence de *rate limiting* ou de mesures spécifiques contre les injections SQL au-delà de l'utilisation systématique de paramètres positionnels via GORM.

9.3 Axes d'amélioration (temps réel WebSocket, scalabilité, tests e2e, durcissement sécurité, licence)

Breezy est aujourd'hui un socle solide et pleinement opérationnel, prêt à évoluer. Pour aller encore plus loin, nous avons déjà identifié plusieurs pistes d'amélioration pour les prochaines étapes. L'idée est d'enrichir l'expérience utilisateur, notamment en intégrant des WebSockets pour rendre les notifications et les échanges par messagerie instantanés. Côté technique, nous prévoyons de faire monter en gamme notre infrastructure en adoptant des outils d'orchestration comme Kubernetes pour faciliter la gestion de la charge. Enfin, nous souhaitons consolider la fiabilité de l'application en renforçant notre couverture de tests, particulièrement sur les scénarios de bout en bout, et en durcissant nos protocoles de sécurité. L'objectif est clair : garantir que Breezy reste une plateforme pérenne, performante et capable de grandir sereinement.

10. Conclusion

Ce projet nous a permis de concevoir et de développer une application web complète, structurée autour d'un frontend Angular moderne et d'un backend Go découpé en plusieurs services spécialisés. L'objectif initial était de réaliser un réseau social fonctionnel répondant aux fonctionnalités attendues du sujet, mais le projet a progressivement évolué vers une application réellement exploitable.

Sur la partie frontend, nous avons mis en place une interface développée avec Angular, organisée autour de pages, de composants réutilisables, de services métier et de guards de navigation. Cette architecture nous a permis de séparer clairement l'affichage, la logique applicative et les échanges avec l'API. Les services Angular centralisent les appels HTTP vers le backend, tandis que les intercepteurs gèrent les aspects transverses comme l'authentification, le rafraîchissement de session, les erreurs globales et les états de chargement.

Le frontend intègre également des éléments d'expérience utilisateur importants, comme le thème, le mode invité, les messages d'erreur, les modales, les composants de posts, les profils, les notifications et la messagerie.

Sur la partie backend, nous avons fait le choix d'une architecture découpée en microservices Go. Chaque service porte une responsabilité précise : authentification, utilisateurs, profils, posts, médias, modération, notifications, messages et relations sociales. L'ensemble est exposé au frontend via une API Gateway, qui joue le rôle de point d'entrée unique. Cette organisation rend l'application plus lisible, plus maintenable et plus proche d'une architecture professionnelle. Elle permet également de masquer au frontend la complexité interne du backend : le navigateur communique simplement avec l'api, puis la gateway redirige les requêtes vers le bon service.

Les fonctionnalités obligatoires du sujet ont été couvertes. Ces fonctionnalités ne sont pas seulement présentes en surface, elles sont reliées à une vraie logique métier backend, à une base de données structurée et à une interface utilisateur cohérente.

Nous sommes également allés plus loin que les attentes initiales. L'application intègre notamment une messagerie privée, des sondages, des reposts, des bookmarks, des profils privés, un système de blocage, les mentions, la traduction, ainsi qu'un module de modération complet. Ces ajouts ont renforcé la richesse fonctionnelle du projet.

Le projet a aussi été pensé pour le déploiement. L'application est conteneurisée avec Docker Compose, ce qui permet de lancer le frontend, l'API Gateway, les services backend, la base PostgreSQL et les outils annexes dans un environnement cohérent. Le frontend est servi par Nginx, qui redirige les appels api vers la gateway. Le site a été mis en production, ce qui représente une étape importante : le projet n'est pas resté limité à un fonctionnement local, mais a été déployé dans un environnement accessible, avec une configuration adaptée à un usage réel.

Cette mise en production nous a permis de prendre en compte des problématiques concrètes souvent absentes d'un simple développement local : configuration des variables d'environnement, proxy HTTP, gestion des ports, séparation entre services internes et services exposés, persistance des données, build frontend, configuration Nginx et cohérence entre environnement de développement et environnement de production.

En conclusion, ce projet répond aux exigences du sujet tout en les dépassant largement. Les choix techniques réalisés, aussi bien côté frontend que côté backend, montrent une volonté de produire une solution robuste et évolutive. La mise en production finalise cette démarche en transformant le projet en une application réellement exploitable, et non seulement en une démonstration technique locale.

11. Bibliographie et webographie

1. Langages et Frameworks

Go (Golang) : Documentation officielle du langage <https://go.dev/doc/>

Gin Gonic : Documentation du framework web pour Go <https://gin-gonic.com/docs/>

GORM : Documentation de l'ORM utilisé pour PostgreSQL <https://gorm.io/docs/>

Angular : Documentation officielle du framework frontend <https://angular.dev/>

2. Architecture et Infrastructure

Docker : Documentation officielle pour la conteneurisation <https://docs.docker.com/>

PostgreSQL : Documentation officielle du système de gestion de base de données
<https://www.postgresql.org/docs/>

Nginx : Documentation du serveur web et reverse proxy <https://nginx.org/en/docs/>

3. Sécurité et Protocoles

JWT (JSON Web Tokens) : Introduction et spécifications <https://jwt.io/introduction>

OAuth 2.0 : Documentation et guides d'implémentation <https://oauth.net/2/>

Bcrypt : Documentation sur l'algorithme de hachage de mots de passe
<https://pkg.go.dev/golang.org/x/crypto/bcrypt>

Let's Encrypt : Documentation pour la gestion des certificats HTTPS
<https://letsencrypt.org/docs/>

12. Annexes

Les annexes sont disponibles sur github :

https://github.com/achour14/projet_web_groupe2_cesi/tree/main/annexes_livable